

DeFi 收益轮动与自动复投回测平台 · 技术报告

课程：金融软件工程 团队：Xingrui Zhao (组长) · Ziqing Ji · Siyi Chen · Jingcheng Wang
日期：2026 · 06 · 12 仓库：https://github.com/Blissar06/241275047_jrrg_exp1 字数统计：约 4,800 字

1 · 问题定义

1.1 背景与挑战

去中心化金融 (DeFi) 生态中，**收益池 (Yield Pool)** 数量爆炸性增长：以 DefiLlama 截至 2026 年 5 月的全网数据为例，仅 Ethereum 主网就有超过 1,000 个活跃池，覆盖借贷协议 (Aave、Compound、Maple)、流性质押衍生品 (Lido stETH、Rocket Pool rETH、ether.fi weETH)、收益聚合器 (Yearn、Convex) 等多种范式。这些池子的年化收益率 (APY) 从稳定币池的 4 ~ 8% 到 ETH 衍生品池的 2.5%、再到部分新兴池的 50%+ 不等，且**每个池的 APY 不是常量，会随 TVL 变化、利用率波动、市场行情等因素实时漂移**。

对于一个手持 DeFi 资产的用户，他面临一个本质上的**动态最优化问题**：

- 当前应该把资金放在哪个池？
- 什么时候应该把资金从一个池**轮动**到另一个池？
- 何时应该把累积的 yield reward 重新投入 (**复投**) 以触发复利效应？
- 这些操作的**摩擦成本** (Gas + 滑点 + LVR) 是否值得？

人工判断这些问题需要每天看大量数据，**几乎不可量化**；直接在链上试错的代价是真金白银——每次调仓的 Gas 费、滑点损失、可能触发的 LVR (Loss-versus-Rebalancing)。

1.2 项目目标

本项目的目标是构建一个**离线回测平台**，在不接触链上交易的前提下：

1. **量化**多池轮动 + 自动复投策略在历史/真实数据上的实际收益
2. **拆解**收益不及理论上限的原因 (摩擦成本占比、调仓滞后折损)
3. **对比**不同策略参数下的风险收益特征 (年化收益、最大回撤、Sharpe 比率)
4. **回答业务问题**——“滑点如何随池深变化？”“频繁调仓是否真的更优？”

通过这个平台，DeFi 策略研究员可以在不冒真金白银风险的情况下迭代策略；学术研究者可以系统化研究 DeFi 摩擦成本对 LP 实际收益的侵蚀程度；普通用户可以理解为什么“看着收益高的池子”未必赚钱。

1.3 与同类工作的区别

现有 DeFi 回测工具大多聚焦于**单一协议内的策略** (如 Uniswap V3 的 LP 区间策略)，或者**仅用合成数**

据做学术性验证。本项目的差异化定位是：

- **跨协议、跨池的轮动决策**——而不仅是单池内的微观策略
- **同时使用合成数据 + 真实链上数据**——通过 DefiLlama API 直接拉取历史 APY、TVL、token price
- **完整的归因分解**——把"理论收益 - 实际收益"的差额拆解到 Gas、滑点、LVR、调仓空窗 4 个可解释来源

2 · 数学模型

本系统的核心数学模型由 **5 组公式** 构成。所有公式实现时**全程使用 Python Decimal 类型（28 位精度）**，不使用 IEEE 754 浮点数，以避免误差累积。

2.1 收益拥挤衰减（Capacity Decay）

当用户把规模为 `Capital` 的资金注入 TVL 为 `TVL` 的池时，池内年化收益率会被稀释：

```
APY_actual = APY_nominal * TVL / (TVL + Capital)
```

物理意义：池总收益分摊到所有 LP，用户份额是 $\text{Capital} / (\text{TVL} + \text{Capital})$ 。该公式在 `data_model/preprocessor.py:apply_capacity_decay` 实现。

针对借贷池，进一步引入利用率 U 的二阶段修正：

```
U < 0.8 (kink 之下): APY_eff = APY_base * (1 - U * 0.1)
U ≥ 0.8 (kink 之上): APY_eff = APY_base ÷ (1 + 5 * (U - 0.8))
```

体现 Aave-style 利率模型中 kink 点之上的边际成本超线性上升。

2.2 多因子综合评分（FR-03）

对每个候选池 p ，综合得分为 6 个因子的加权和（每个因子先各自跨池做 z-score 归一化）：

```
Score(p) = w_mom * Momentum(p)
          + w_vol * VolPenalty(p)
          + w_mdd * MDDPenalty(p)
          + w_cara * CARA(p)
          + w_pvol * PriceVolPenalty(p)
          + w_pmdd * PriceMDDPenalty(p)
```

其中：

- **动量因子**： $\text{Momentum}(p) = \text{EWMA}_\lambda(\text{apy_series}(p))$ ，递推式 $M_t = \lambda \cdot M_{t-1} + (1-\lambda) \cdot$

$r_t, \lambda \in (0, 1)$

- **下行波动惩罚**： $\text{VolPenalty}(p) = \text{std}(\{d_i : d_i < 0\})$ ， d 是 APY 差分
- **最大回撤惩罚**： $\text{MDDPenalty}(p) = \max(1 - \text{apy}_t / \text{cummax}(\text{apy}[0:t]))$
- **CARA 效用调整**： $\text{CARA}(p) = -\exp(-\alpha \cdot r_p)$ ， $\alpha > 0$ 为风险厌恶系数
- **价格风险**： $\text{PriceVol}/\text{MDDPenalty}$ 把上述 Vol/MDD 公式应用到 `token_price_series` 而非 APY series

权重需归一化： $\sum w_i = 1$ 。在 `strategy/scoring_engine.py:ScoringEngine.run` 中实现。

2.3 门槛约束轮动 (FR-04)

每个 tick t ，引擎对当前持仓做决策：是否轮动到 top-1 候选池？必须**同时满足**两个条件：

条件 ① τ -reset 偏离度检验：

`score_gap(top, current) >  $\tau$`

或 `|APY_target - APY_current| / |APY_current| >  $\tau$`

条件 ② 双门槛检验：

`expected_gain  $\geq$  friction_cost + threshold  $\times$  principal`

其中：

- `expected_gain` 由 `APYDeltaGainEstimator` 估算，公式为 `gain = invested  $\times$  (APY_target - APY_current + price_drift)  $\times$  horizon / 365`
- `friction_cost = gas + slippage + lvr`
- `threshold` 是用户配置的本金净增益占比阈值（默认 0.1%）

两个条件任一不满足 $\rightarrow$ HOLD 决策；都满足 $\rightarrow$ ROTATE。该判定在 `strategy/rotation_engine.py:RotationEngine.evaluate` 实现，状态机有 7 个状态（IDLE/SCORING/RANKED/EVALUATING/COMMITTING/HOLDING/ERROR）。

2.4 Mark-to-Market 重估 ☆

每个 tick，按持仓池的 `token_price` 比例重估持仓的法币价值：

```
ratio = token_price[t] / token_price[t-1]
new_principal = principal  $\times$  ratio
new_pending_reward = pending_reward  $\times$  ratio
```

这一步**直接反映底层资产价格波动对 NAV 的影响**。`backtest/engine.py:_mark_to_market` 实现。该机制是本项目最关键的设计修复——在引入它之前，token 涨跌完全不影响 NAV，导致 Sharpe 出现 1198 这种数学上的伪值；引入后 Sharpe 回到 [0.17, 1.90] 的合理区间（详见第 5 节实验）。

2.5 摩擦成本三分量

Gas 成本

```
gas_cost = (base_fee + priority_fee) × gas_limit_by_op
```

其中 `gas_limit_by_op` 按操作类型查表：ROTATE = 350,000、REINVEST = 180,000、CLAIM = 80,000、DEPOSIT = 200,000。

滑点（阶梯函数）

```
ratio = trade_size / pool_TVL
ratio < 0.01          → slip_rate = 0.1% (low)
0.01 ≤ ratio < 0.05   → slip_rate = 0.3% (mid)
ratio ≥ 0.05          → slip_rate = 0.8% (high)

slippage_cost = trade_size × slip_rate
```

LVR (Loss-versus-Rebalancing)

```
LVR = |oracle_price - pool_price| / oracle_price × trade_size × 0.5
```

物理意义：套利者从 LP 价格滞后中提取的价值。该公式参考 [Milionis et al., 2022, "Automated Market Making and Loss-Versus-Rebalancing"]。

2.6 收益归因守恒等式

整个回测期满足：

```
theoretical_return = actual_return
                    + total_gas_cost
                    + total_slippage_cost
                    + total_lvr_cost
                    + rotation_idle_cost
```

其中 `theoretical_return` 是"每 tick 都持有 max-APY 池且无摩擦"的复利上限，`rotation_idle_cost` 是位于次优池的机会成本。集成测试 `test_attribution_decomposition_conserved` 验证：

```
abs(reconstructed - theoretical) <= Decimal(1)
```

即对 100k 本金，误差 < 1 元 = **0.001%**。

3 · 系统架构

3.1 5 层模块划分



依赖方向严格自上而下，低层不感知高层。每层间通过抽象接口对接：

- IScorer —— 评分器（6 个实现：Momentum / DownsideVolPenalty / MaxDrawdownPenalty / CARAUtility / TokenPriceVolPenalty / TokenPriceMDDPenalty）
- IFrictionEstimator —— 摩擦成本估算（生产实现 + 测试 stub）
- IGainEstimator —— 增益估算（含 price drift 修正）
- IRotationPolicy —— 轮动策略（默认实现：ThresholdRotationPolicy）

这种设计使得任何一个组件都可以热插拔替换，例如研究员可以新增一个 LiquidityConcentrationScorer 而不修改其他代码。

3.2 关键数据结构

所有金融数据结构均为 frozen dataclass + slots=True，保证不可变性（满足 NFR-02 复现性要求）：

```
@dataclass(frozen=True, slots=True)
class PoolMetrics:
    pool_id: str
    apy_series: Tuple[Decimal, ...]      # 回看 APY 序列
    tvl: Decimal
    vol_30d: Decimal
    token_price: Decimal
    gas_base_fee: Decimal
    token_price_series: Tuple[Decimal, ...]  # 回看价格序列

@dataclass(frozen=True, slots=True)
class EnvSnapshot:
    tick: int
    timestamp: datetime
    oracle_price: Mapping[str, Decimal]
```

```

    gas_base_fee: Decimal
    gas_priority_fee: Decimal

@dataclass(frozen=True, slots=True)
class AssetSnapshot:
    tick: int
    pools: Mapping[str, PoolMetrics]
    env: EnvSnapshot

```

容器类型用 MappingProxyType 包装，防止外部修改穿透：

```

def __post_init__(self):
    object.__setattr__(self, "pools",
                        MappingProxyType(dict(self.pools)))

```

3.3 事件流

每个 tick 的处理流程：

```

AssetSnapshot[t]
    ↓
① EventInjector.apply()    → 注入 Gas_Spike / Pool_Exploit / Liquidity_Dryup
    ↓
② _mark_to_market()        → 按 token_price 比例重估持仓
    ↓
③ _accrue_yield()          → pending_reward += principal × APY / 365
    ↓
④ ScoringEngine.run()      → 6 Scorer 并行 → 加权 → 稳定排序
    ↓
⑤ ReinvestEngine.evaluate   → 净效用 > 0 时 commit_reinvest
    ↓
⑥ RotationEngine.evaluate   → τ-reset + gate 全过 时 commit
    ↓
⑦ 写入 nav/trade/score/reinvest 4 张日志
    ↓
AssetSnapshot[t+1]

```

实现位置：backtest/engine.py:BacktestEngine.run。整个流程**严格串行、无并发**，保证相同输入两次运行输出完全一致。

4 · 仿真设计

4.1 时间推进

本系统采用**离散时间步推进**（discrete-time tick stepping），而非事件驱动调度。每个 tick 对应一个真实时间步（默认每日一步，可配置每小时）。这种选择的依据是：

- DeFi 数据原生是离散采样（DefiLlama 提供日级 APY 快照）
- 时间步推进的可复现性远优于事件驱动（无并发顺序歧义）
- 性能开销极低：365 tick × 3 池仅需 0.19 秒

4.2 用户行为建模

本系统**没有显式建模用户**，因为目标是评估**策略本身**而非用户群体行为。RotationEngine + ReinvestEngine 实际上扮演了“理性用户代理”的角色——它们的行为完全由参数化的策略规则决定：

- 给定 `τ-reset = 0.05`、`threshold = 0.001`、`reinvest_window = 30`、`risk_premium_multiplier = 1.5`，引擎在每个 tick 给出确定性决策
- 5 个内置策略预设（保守稳健 / 均衡 / 激进动量 / 低频价值 / 极端风险厌恶）覆盖了从风险厌恶到风险偏好的全象限

4.3 随机性控制

为满足 **NFR-02 结果绝对复现**要求，所有随机性都通过 `numpy.random.default_rng(seed)` 加种子控制：

- 合成数据生成器 `data/sample_data.py:generate_sample_data(seed=42)`
- 属性测试中 `hypothesis` 自动按 `@settings(max_examples=50)` 重放反例

集成测试 `test_two_runs_produce_identical_outputs` 验证：相同种子两次 run 的 `nav_log / trade_log / score_log` 三张 DataFrame **逐元素相等**（`DataFrame.equals` 返回 True）。

4.4 压力事件注入

`backtest/event_injector.py:EventInjector` 实现 3 类压力事件：

```
class StressEvent:
    event_type: EventType          # GAS_SPIKE / POOL_EXPLOIT / LIQUIDITY_DRYUP
    start_tick: int
    duration: int
    impact_ratio: Decimal
    target_pool_id: Optional[str]
```

- **GAS_SPIKE**：`env.gas_base_fee × (1 + impact_ratio)`
- **POOL_EXPLOIT**：目标池末项 APY × `(1 - impact_ratio)`，TVL 同步缩减
- **LIQUIDITY_DRYUP**：目标池 TVL × `(1 - impact_ratio)`，APY 不变

`apply()` 方法返回**新的 AssetSnapshot**——不修改原对象。

5 · 实验与验证

本项目设计了 **325 个自动化测试**，分布在 4 类目录：

- tests/blackbox/ 88 个 —— 按测试方法学（等价类/边界值/决策表/场景）
- tests/whitebox/ 41 个 —— 路径覆盖 + 条件组合
- tests/property/ 11 个（每个含 50+ 随机用例）—— hypothesis 属性测试
- tests/perf/ 10 个 —— pytest-benchmark 性能基准
- 其余 175 个为按模块组织的单元测试

总分支覆盖率 **87.5%**，核心策略模块超过 90%。

5.1 基础功能测试

测试 1：守恒等式

```
def test_attribution_decomposition_conserved(snapshots):  
    ...  
    reconstructed = actual + gas + slippage + lvr + idle  
    diff = abs(reconstructed - theoretical)  
    assert diff <= Decimal(1)    # 误差 < 1 元 / 100k
```

实测误差通常在 1e-4 量级，远低于断言阈值。

测试 2：复现性（NFR-02）

```
@given(seed=st.integers(min_value=1, max_value=10000))  
def test_PROP_DET_01_same_synthetic_seed_same_nav(seed):  
    snaps = build_snapshots(generate_sample_data(seed=seed))  
    r1, r2 = engine.run(snaps), engine.run(snaps)  
    assert r1.nav_log["nav"].equals(r2.nav_log["nav"])
```

50 次随机种子 + 2 次重放，全部通过。

测试 3：性能基准（NFR-04）

```
PERF-RUN-01  365 tick × 3 池    → median 188 ms  
PERF-RUN-02  1000 tick × 5 池   → median 747 ms
```

线性外推到 10,000 tick × 10 池约 15 秒，仍低于 5 秒目标但在可接受范围（属于本项目降级项之一）。

5.2 极端场景测试

场景 1：Pool_Exploit 后被攻击池评分下跌

```
def test_SC_EXPL_01_target_pool_score_drops_after_event(snapshots):
    injector = EventInjector([
        StressEvent(EventType.POOL_EXPLOIT, start_tick=200,
                    duration=1, impact_ratio=Decimal("0.9"),
                    target_pool_id="pool_B"),
    ])
    result = engine.run(snapshots)
    s_before = result.score_log.query("tick==199 and pool_id=='pool_B'")["total_score"].
    s_after = result.score_log.query("tick==200 and pool_id=='pool_B'")["total_score"].
    assert s_after < s_before
```

实测：-0.222 → -0.977，下跌 0.755 个 z-score 单位。

场景 2：Gas Spike 期间调仓抑制

```
def test_SC_GAS_02_gate_fails_more_during_spike():
    spike_count = rotates_in_window(spike_run, window=(100, 129))
    normal_count = rotates_in_window(normal_run, window=(100, 129))
    assert spike_count <= normal_count
```

注入 20× Gas spike 后，窗口内调仓数从 14 次降至 2 次，符合理性策略应有的行为。

5.3 对比分析与结论洞察

5 个策略预设在同一份合成数据（365 tick × 3 池）上的实测结果：

策略	年化	MDD	Sharpe	调仓数
保守稳健 ✓	+ 8.82%	5.80%	1.076	19
极端风险厌恶 ✓	+ 8.36%	5.80%	1.020	19
均衡（默认）	-3.65%	20.69%	-0.230	14
激进动量 ✗	-6.50%	20.44%	-0.367	1
低频价值 ✗	-11.71%	21.57%	-0.781	16

3 个可解释洞察：

- 滑点 vs 池深：Maple USDC（TVL \$3.27B）每次调仓滑点 0.1%，Vesper ETH（TVL \$5.2M）滑点 0.3% → 大 TVL 池可节省 67% 滑点成本。这是 FrictionEstimator 阶梯函数的直接体现。
- 价格风险评分至关重要：不考虑 token_price 风险的「激进动量」策略亏损 6.5%，加入 TokenPriceVolPenalty + TokenPriceMDDPenalty 的「保守稳健」反而赚 8.8%。评分维度的设计决定了策略生死。

3. 调仓数量不是关键，调仓时机才是：激进 1 次调仓亏损、低频 16 次调仓亏损、保守 19 次调仓盈利。盈亏关键在于是否在价格风险升高时及时撤离，而非调仓频率本身。

5.4 真实链上数据 Case Study

数据：DefiLlama 真实历史（Maple USDC / Vesper ETH / Lido stETH × 300 天）

保守稳健策略实测结果：

- 年化收益 +8.74%
- 最大回撤 2.15%
- Sharpe 1.540
- 调仓 2 次
- USDC 占比 99%

模型正确识别 stETH 价格风险（历史价格 0.53 ~ 1.38 大幅波动），把绝大部分时间留在 Maple USDC 稳定币池——这不是人工告诉策略的，是评分器自己判断的。

6 · 团队分工与反思

6.1 4 人分工（按模块边界 · 平均 25%）

成员	负责模块	关键产出
Xingrui Zhao（组长）	data_model/ · data/ onchain_fetcher.py · 文档	CSV/Parquet 加载、DefiLlama API 接入、frozen dataclass、需求与设计文档
Ziqing Ji	strategy/（6 Scorer + 引擎 + 5 预设）	MomentumScorer、Vol/MDD/CARA、TokenPrice 双 Scorer、RotationEngine 7 状态机、ReinvestEngine
Siyi Chen	backtest/ · report/	FrictionEstimator 三分量、EventInjector 3 事件、Mark-to-Market 引擎、归因守恒分解
Jingcheng Wang	ui/ · tests/ · CI	Streamlit 6 Tab、Plotly 10 图表、325 测试矩阵、GitHub Actions CI

协作机制：

- Git 分支管理 + PR 互审，避免主分支被未审阅代码污染
- 接口先行——IScorer / IFrictionEstimator 等抽象接口在编码前就由组长定稿，4 人并行开发
- 每周一次集成测试，CI 自动跑 325 测试
- 关键决策（如 MTM 重构）召集全员讨论，避免局部最优

6.2 项目难点

难点 1：Sharpe 异常飙升 → Mark-to-Market 重构

最初模型未对持仓做 mark-to-market，token 价格波动完全不传导到 NAV，导致：

- Sharpe = **1198.376**（数学上"无风险高收益"）
- MDD = **0.00%**（NAV 单调上升）
- 所有 5 个策略预设跑出**完全相同**的结果

定位过程：在端到端测试通过后才发现 Sharpe 数值离谱。回头审视 `BacktestEngine.run` 主循环，确认了"持仓只按 APY 累计、忽略 token 价格变化"这一设计缺陷。**重构耗时 2 轮迭代**，涉及：

1. 在 `BacktestEngine` 增加 `_mark_to_market(prev_snap, curr_snap)` 步骤
2. 给 `data/sample_data.py` 加 `token_price` 几何 Brownian 路径
3. 给 `PoolMetrics` 新增 `token_price_series` 字段
4. 新增 `TokenPriceVolPenaltyScorer + TokenPriceMDDPenaltyScorer`
5. 接入 `DefiLlama` coins API 拉真实价格

重构后 Sharpe 回到 `[0.17, 1.90]`，5 策略产生 5 种结果。

难点 2：DefiLlama 跨池时间戳对齐

`DefiLlama` 的 `/chart/{pool_id}` 端点返回的时间戳带秒级偏移（每池快照时间不同），导致跨池 `merge_asof` 后大量 NaN。**修复方式**：在 `fetch_pool_history` 中加 `dt.floor("D")` 把时间戳归一化到日级别，并去重保留末条。

难点 3：决策表 / 条件组合用例设计

按课程方法学，需要给 `RotationEngine.evaluate` 这种多条件判定建决策表。需要先做控制流图分析、列出所有条件组合、然后构造能精准触发每一行的测试输入。**这是机械性强但智力密度高的工作**，耗时约 1 天完成 12 个决策表用例 + 19 个条件组合用例。

难点 4：属性测试发现的隐藏 bug

`hypothesis` 在跑 `test_PROP_DET_01` 时自动暴露了一个 bug：当 `n_days < crash_window` 时，`sample_data.py` 的切片操作 `apy_path[200:205]` 在长度 60 的数组上产生空切片，与广播操作冲突。**这个 bug 人工写测试 100% 想不到**，是属性测试的直接价值。

6.3 改进方向

如果时间允许，可以进一步：

1. **多链支持**——目前只接入了 Ethereum 主网，未来可接 Arbitrum / Polygon / Optimism
2. **LP 双代币建模**——目前所有池都按单代币近似，不能准确建模 Uniswap V3 等双代币 LP
3. **强化学习驱动策略**——用 DQN/PPO 端到端学习轮动决策，以本系统为 baseline

4. **突变测试**——用 mutmut 度量测试套件本身的强度（catch 率）
5. **真实 Gas 价格历史**——目前 Gas 数据是合成的，未来可接 Etherscan API 拿真实历史

7 · 演示材料

7.1 可执行 Demo

提供 **4 个一键脚本**，无需复杂配置：

```
# 一键 1：跑全部 325 测试，验证装机正确
pytest tests/ --ignore=tests/perf

# 一键 2：端到端示例 + 7 项自动验收断言
python run_example.py

# 一键 3：拉取真实 DefiLlama 数据（自动选 demo 三池）
python fetch_data.py --demo --days 300

# 一键 4：启动 Streamlit 交互看板
streamlit run ui/app.py
```

预期输出：

- 一键 1：325 passed in 30.46s
- 一键 2：末尾 —— 全部验收通过 ——
- 一键 3：[save] pool CSV → data/real_pools.csv (900 rows, 3 pools)
- 一键 4：浏览器自动打开 <http://localhost:8501>，看板含 6 Tab × 10 类图表

完整复现流程见 [docs/INSTALL_GUIDE.md](#)，目标耗时 **30 分钟内跑通 Demo**。

附录 · 引用

- Milionis et al., 2022. "Automated Market Making and Loss-Versus-Rebalancing." arXiv:2208.06046.
- DefiLlama. Yields API & Coins API. <https://defillama.com/docs/api>
- Streamlit Inc. Streamlit Documentation. <https://docs.streamlit.io/>
- pytest dev. pytest Documentation. <https://docs.pytest.org/>

仓库：https://github.com/Blissar06/241275047_jrrg_exp1 **字数统计（不含代码块）**：约 **4,800 字**（≥ 3,000 字要求）